

CS 471 Team 2 Final Project Progress Report

Names:

Shane Dyrdahl, Alastair Raymond, Riley Saliba, Dax Taraleskof

Date:

12 November 2024

Motivation:

As mentioned in the project proposal, our intention behind this project is to create a model through machine learning that looks through a dataset of bank transactions and determines if they're fraud or not. Our reasoning for choosing this dataset was because we were looking for a dataset which had a high amount of features and the features looked like there was a high degree of correlation between them. We figured there was good potential with correlating between certain features of the dataset like the initial balance vs the new balance after transaction.

Dataset:

Our dataset is the fraudulent transactions dataset

(<https://www.kaggle.com/datasets/vardhansiramdasu/fraudulent-transactions-prediction>)

These are the features of our dataset:

- step
- type (cash-in, cash-out, debit, payment, transfer)
- amount (amount of the transaction)
- nameOrig (ID for customer who made the transaction)
- oldbalanceOrg (initial balance before transaction)
- newbalanceOrig (new balance after transaction)
- nameDest (ID for customer who is the recipient of the transaction)
- oldbalanceDest (initial balance recipient before the transaction)
- newbalanceDest (new balance recipient after the transaction)
- isFraud (transactions made by fraudulent agents)
- isFlaggedFraud (flags illegal attempts. An illegal attempt in this dataset is an attempt to transfer more than 200.000 in a single transaction.)

Through our current rules based approach, the only features we utilize so far are "amount" and "isFraud"

When looking at prior datasets, we encountered a few that had columns with data that may have had to be parsed due to there being more than a single element in that

column (multiple genres for a movie dataset). That would have been an inconvenience to handle, so this dataset suits us better as each column only contains one element.

The main issue is that the data is significantly imbalanced as there are about 6,084,104 non-fraudulent transactions and 6,485 fraudulent transactions. Therefore, we had to utilize oversampling so that fraudulent transactions are equally represented.

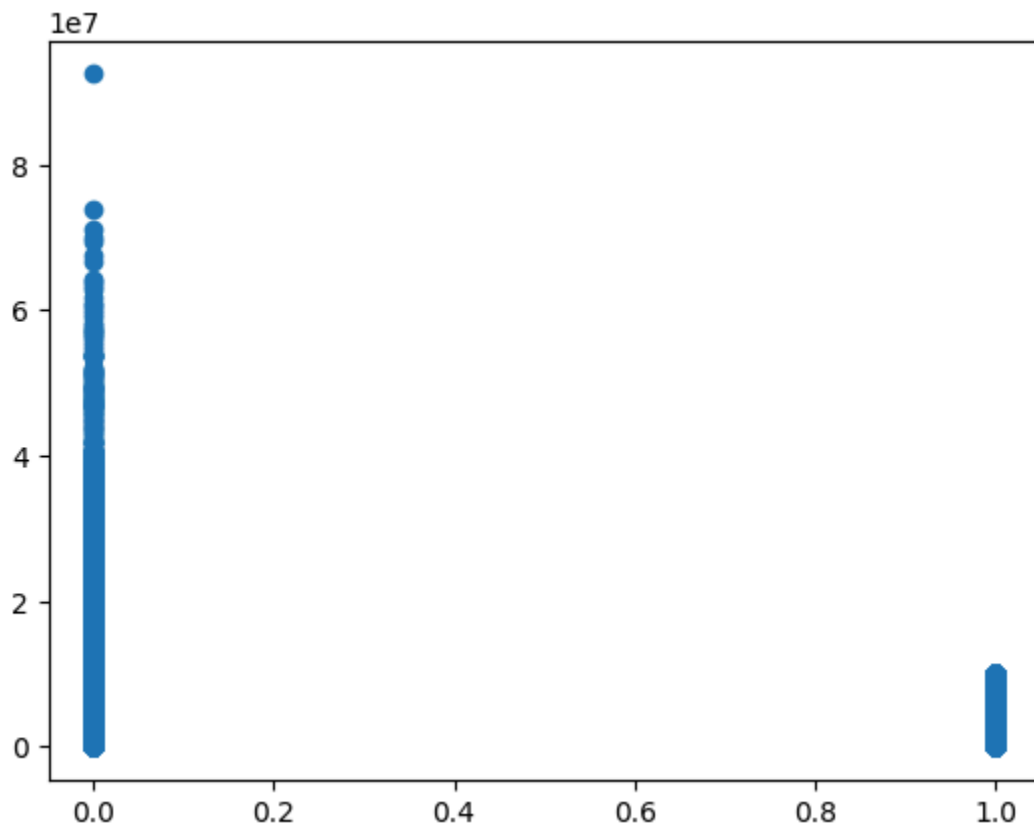
Exploratory Data Analysis:

This is the code we ran for visualization:

```
# visualization (takes a while)
pyplot.scatter(dataset[['isFraud']], dataset[['amount']])
pyplot.show()
```

'isFraud' represents the x-axis where 1 means fraud and 0 means not fraud
'amount' represents the y-axis data to show the amount of the transaction

When performing a visualization of the dataset through a scatter plot, we get this:



As stated prior, the dataset was initially heavily imbalanced and this graph shows that a majority of the data is 0 (non-fraud) and a minority was 1 (fraud)

Here is the code we ran to generate the correlation matrix:

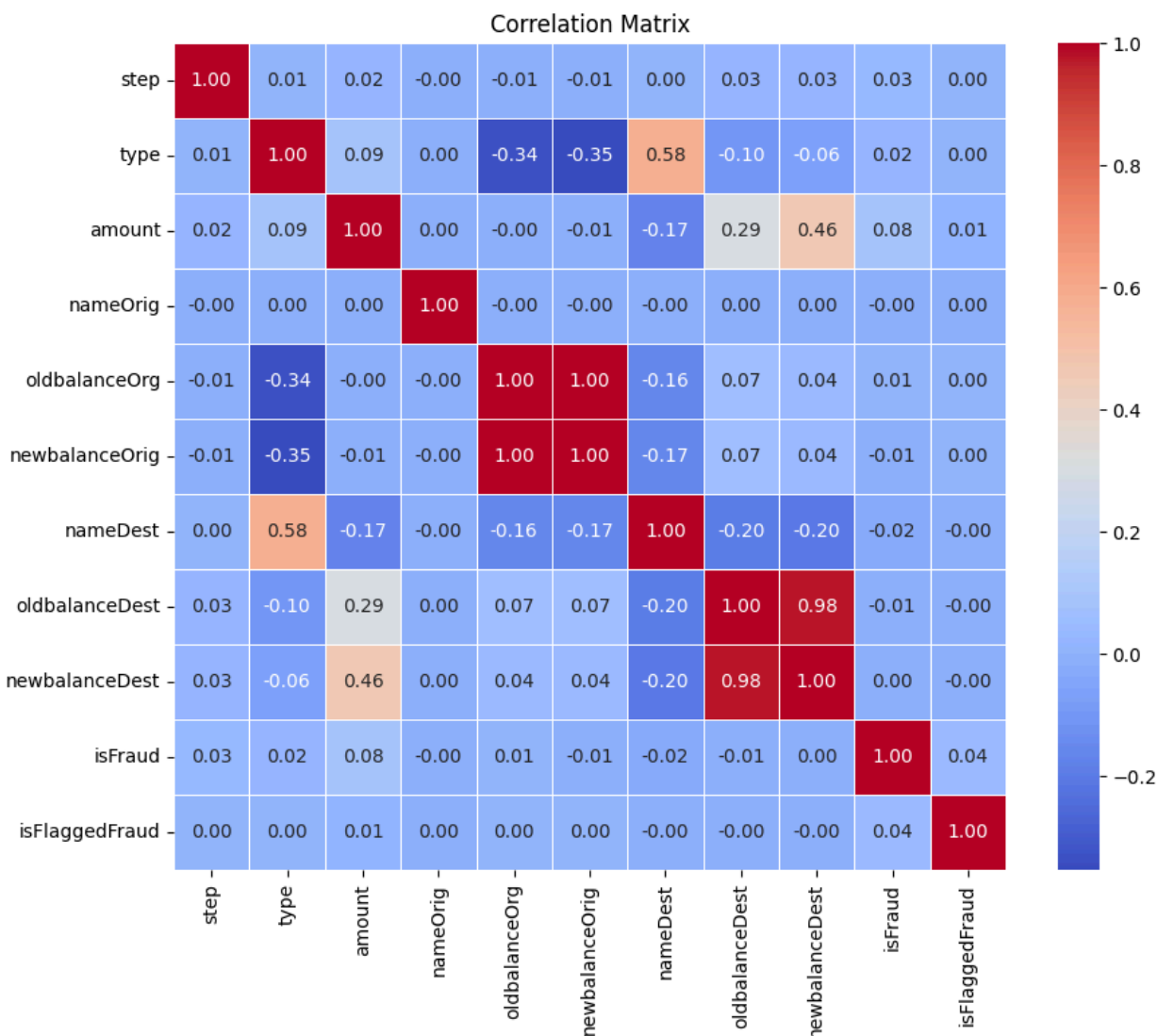
```
# Correlation matrix
plt.figure(figsize=(10, 8))
correlation_matrix = dataset.corr()
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm', fmt='.2f',
linewidths=0.5)
plt.title('Correlation Matrix')
plt.show()
```

We first initialize a figure with the size set to 10 by 8 inches

Then we calculate the correlation matrix based on the dataset

Next, the heatmap is then generated for the correlation matrix by first adding which matrix data should be used, determining true for displaying the values in each cell, indicating the color map to be used for the heatmap, formatting the annotations of each cell to be by two decimal places, and finally formatting the lines between the cells

This is a correlation matrix generated for our dataset:



There appears to be a strong correlation between oldbalanceOrg and newbalanceOrig
Also, there's fairly high correlation between transaction type and nameDest

Pre-processing:

Since we're using a machine learning model, numerical inputs would be vastly preferred so we use encoding to convert the categorical variables (type, nameOrig, and nameDest) each into unique integers as these were the only labels that were strings

```
# encode strings
label_encoder = preprocessing.LabelEncoder()
one_hot_encoder = preprocessing.OneHotEncoder()
```

```
dataset['type'] = label_encoder.fit_transform(dataset['type'])
dataset['nameOrig'] = label_encoder.fit_transform(dataset['nameOrig'])
dataset['nameDest'] = label_encoder.fit_transform(dataset['nameDest'])
```

We separate the features from the target variable so that the independent variables of X can be used to make predictions while the dependent variable of Y is the outcome that the model uses to predict with. In this case, the variables in X are used to predict if a transaction is fraud or not

```
X = dataset[['type', 'amount', 'nameOrig', 'oldbalanceOrg',
'newbalanceOrig', 'nameDest', 'oldbalanceDest', 'newbalanceDest']]
y = dataset[['isFraud']]
```

In here, we split the data into training and testing

```
# split dataset into training and testing
training_X, testing_X, training_y, testing_y = train_test_split(X, y,
test_size = 0.20)
```

We then further split the training data so that the split is now 60% training, 20% evaluation/validation, and 20% testing

```
training_X, evaluation_X, training_y, evaluation_y =
train_test_split(training_X, training_y, test_size = 0.20)
```

To handle the imbalance nature of the original dataset, we oversample the training data

```
temp_X, temp_y = SMOTE(random_state=130).fit_resample(training_X,
training_y)
training_X = temp_X
training_y = temp_y
```

Algorithm 1 (Rules-based approach):

- Considering we're dealing with amount transactions where there may be certain outliers in the dataset like amounts that are vastly larger or smaller than others, we opted for using the median to sort out the values and find a "typical" amount for transactions
- This median value gives us a general idea for what we should set the threshold value to when comparing transaction amounts with the threshold to predict if they're fraud

```
# calculate statistics
median = float(training_X[['amount']].median().iloc[0])
```

For the actual rules based approach, this is done within the evaluate function:

```
def evaluate(threshold, data, actual):
    false_negative_count = 0
    true_positive_count = 0
    merged_data = data.copy()
    merged_data.insert(8, "isFraud", actual, True)
    potential_fraud = merged_data.loc[(merged_data['amount'] < threshold)]
    actual_fraud = potential_fraud.loc[(potential_fraud['isFraud'] == 1)]
    not_fraud = potential_fraud.loc[(potential_fraud['isFraud'] == 0)]
    true_positive_count = len(actual_fraud)
    total_fraud_count = len(merged_data.loc[(merged_data['isFraud'] ==
1)])
    false_negative_count = total_fraud_count - len(actual_fraud)
    false_positive_count = len(not_fraud)

    # calculate recall metric
    recall = (true_positive_count / (true_positive_count +
false_negative_count))
    print("Recall: " + str(recall))

    # calculate precision metric
    precision = (true_positive_count / (true_positive_count +
false_positive_count))
    print("Precision: " + str(precision))

    # calculate F1-score
    print("F1-Score: " + str(2 * ((precision * recall) / (precision +
recall))))
```

- The threshold input parameter is the amount that we compare to the transaction to determine if it's fraudulent or not
- data parameter is the dataset with the transaction info and actual is the label indicating if the transaction is fraud or not (1 for fraud and 0 for not fraud)
- These are counters for true positives (fraud transactions correctly flagged fraud) and false negatives (fraud transactions incorrectly flagged as not fraud)

```
false_negative_count = 0
true_positive_count = 0
```

- This line makes the dataset copy only include transactions that are less than the threshold

```
potential_fraud = merged_data.loc[(merged_data['amount'] < threshold)]
```

- actual_fraud further filters the dataset by only having transactions that actually are labeled as fraud and stores them in actual_fraud variable
- not_fraud does the opposite by filtering to only include transactions that are not fraud

```
actual_fraud = potential_fraud.loc[(potential_fraud['isFraud'] == 1)]  
not_fraud = potential_fraud.loc[(potential_fraud['isFraud'] == 0)]
```

- This counts the number of fraud transactions correctly flagged as fraud

```
true_positive_count = len(actual_fraud)
```

- total_fraud_count is the total amount of fraud transactions in merged_data (copy of the dataset)

```
total_fraud_count = len(merged_data.loc[(merged_data['isFraud'] == 1)])
```

- false negatives are the number of transactions that are fraud but were incorrectly predicted as non fraud. Calculated through subtracting the total fraud transactions in the dataset by the true positives
- false positives are transactions flagged as fraud but aren't actually fraud

```
false_negative_count = total_fraud_count - len(actual_fraud)  
false_positive_count = len(not_fraud)
```

Pros of our rules based approach:

- **Simple and interpretable:** It's easy to implement as it's just a simple comparison between the transaction amount and the adjustable threshold. It's easy to understand why a transaction was flagged as fraud, because it may be over or under a certain amount to be labeled as fraud
- **Computationally inexpensive:** Our dataset is large, but since we're only doing a simple comparison between two values, it's efficient and fast to execute
- **Good recall score:** Our rules-based algorithm scores highly on recall (>0.96). This is beneficial as it means cases of fraud are very unlikely to be missed.

Cons of our rules based approach:

- **Potential to miss flagging as fraud:** detecting fraud transactions can be more complex than just seeing if the amount was above a certain amount as there may be certain use cases where an amount was indeed below the threshold, but was actually a fraud transaction
- **Lower Precision:** After our initial test, we got a precision score of 0.4921; meaning that roughly 49.2% of transactions flagged as fraud were actually fraud.

While 50.8% of transactions are actual transactions incorrectly flagged as fraud.
This means that our model a lot of the time flags actual transactions as fraud